



UNIVERSITY OF KLAGENFURT

CONTENT-BASED VIDEO RETRIEVAL SYSTEM

Implementation and Code Documentation

TEAM MEMBERS: MAHMOUD HAMED, KIRILL FEIGELMAN, ANDRII
ZHUKOV, BEGENCH BATYROV

18.07.2024

1 Introduction

This document provides a detailed analysis and overview of a project designed to process video files, detect scenes, extract keyframes, generate captions, and store this information in an SQLite database. The project comprises several interconnected components: a Flask web application, an HTML front-end, and a video processing script. Each component is described in detail below, followed by an explanation of how they work together to achieve the project's goals.

2 Component Overview

2.1 Flask Web Application (`app.py`)

- **Main Functionality:** This is the core file of the Flask web application.
- **Primary Purpose:** It serves as the entry point for the web server, defining routes for serving HTML pages, keyframes, videos, and handling search queries.
- **Key Routes:**
 - `/`: Serves the `index.html` page.
 - `/keyframe/<path:filename>`: Serves keyframe images from the `keyframes` directory.
 - `/videos/<path:filename>`: Serves video files from the `E:/V3C1-100` directory.
 - `/query`: Handles POST requests to perform search queries on the database.

2.2 HTML Front-End (`index.html`)

- **Main Functionality:** This is the main HTML file that provides the user interface for the web application.
- **Primary Purpose:** It provides a user-friendly interface for entering search queries and displaying search results, including video playback.
- **Key Elements:**
 - `input[type="text"]`: Allows users to enter search queries.
 - `button`: Triggers the search function.
 - `div.result`: Displays the search results, including video details and playback.

2.3 Video Processing Script (`source_code.py`)

- **Main Functionality:** This script processes video files to detect scenes, extract keyframes, generate captions, and store this information in an SQLite database.
- **Key Components and Functions:**
 - **Import Libraries and Initialize Models:**
 - * Various libraries are imported for tasks such as video processing (`ffmpeg`, `cv2`), image handling (`PIL`), scene detection (`TransNetV2`), caption generation (`BLIP`), and embedding generation (`SentenceTransformer`).
 - * The models (`TransNetV2`, `BLIP`, and `SentenceTransformer`) are initialized and loaded.

- **Database Functions:**

- * `create_database(db_path)`: Creates an SQLite database with tables for storing video metadata (`videos`) and scene information (`scenes`). It also creates indexes to optimize queries.
- * `insert_video(conn, video_id, video_path)`: Inserts a new video record into the `videos` table.
- * `insert_scene(conn, video_id, scene_start, scene_end, keyframe_url, caption)`: Inserts a new scene record into the `scenes` table.

- **Video Processing Functions:**

- * `generate_caption(image_path)`: Uses the BLIP model to generate a caption for a given image.
- * `frames_to_timecode(frames, fps)`: Converts frame numbers to timecodes in the format HH:MM:SS.mmm.
- * `get_video_fps(video_path)`: Retrieves the frames per second (FPS) of a video using `ffmpeg`.
- * `extract_frames_as_jpg(video_path, start_time, end_time)`: Extracts frames from a video between specified start and end times and saves them as JPEG images.
- * `select_representative_frame(images, similarity_threshold)`: Selects the most representative frame from a list of images based on perceptual hashing and saliency detection.

- **Scene Detection and Processing:**

- * `process_video(video_path, video_id, parent_folder, conn)`: The main function that processes a video file:
 - Inserts video metadata into the database.
 - Uses the TransNetV2 model to detect scenes and shot boundaries in the video.
 - Extracts frames for each scene and selects a representative keyframe.
 - Generates a caption for the keyframe and inserts scene data into the database.
 - Cleans up temporary files and performs garbage collection.

- **Main Script Execution:**

- The database is created or opened.
- A list of video IDs is defined, and each video is processed sequentially using `process_video`.
- The database connection is closed at the end.

3 Models Used

3.1 TransNetV2

- **Purpose:** Used for scene detection in videos.
- **Usage:** The model is loaded, and its weights are initialized (given the pre-trained weights).

3.2 BLIP (Salesforce/blip-image-captioning-large)

- **Purpose:** Used for generating captions for keyframes.
- **Usage:** The model and processor are loaded and used to generate textual descriptions of images (Keyframes).

3.3 SentenceTransformer (all-MiniLM-L6-v2)

- **Purpose:** Used to generate embeddings for text data, although not explicitly utilized in the current processing function.

4 Databases Utilized

4.1 SQLite Database

- **Purpose:** Stores video and scene information.
- **Tables:**
 - **videos:** Stores video metadata such as video ID and path.
 - **scenes:** Stores scene metadata, including video ID, scene start and end times, keyframe URL, and caption.
- **Indexes:** Created to optimize query performance.

5 Logic Flow

5.1 Initialization

- Libraries and models are imported and initialized.
- Database connection is established, and tables are created if they don't exist.

5.2 Processing Loop

- For each video in the specified list:
 - The video is processed to detect scenes.
 - Frames are extracted, and a representative keyframe is selected.
 - Captions are generated for the keyframes.
 - Scene information is stored in the database.
 - Temporary files are cleaned up.

5.3 Finalization

- The database connection is closed after processing all videos.

6 Specific Algorithms and Processes

- **Scene Detection:** TransNetV2 predicts scene boundaries based on video frames.
- **Frame Extraction:** Frames are extracted using `ffmpeg` and saved as JPEG images.
- **Caption Generation:** The BLIP model generates textual descriptions for images.

- **Representative Frame Selection:** Uses **perceptual hashing** and **saliency detection** to choose the most representative frame from a set of extracted frames.
- **Database Interaction:** SQLite stores and manages video and scene metadata.

7 How It All Works Together

The project consists of a web application and a backend processing system that work together to provide comprehensive video analysis and search functionality:

- **Web Application (Flask and HTML):**
 - The Flask web application (`app.py`) serves the HTML front-end (`index.html`) to users.
 - Users can submit search queries through the web interface.
 - The Flask application handles these queries, processes them using the SentenceTransformer model and retrieves relevant video scenes from the SQLite database.
 - Search results, including video playback and captions, are displayed on the web interface.
- **Backend Processing System:**
 - The video processing script (`source_code.py`) is responsible for analyzing video files, detecting scenes, extracting keyframes, generating captions, and storing all relevant data in the SQLite database.
 - This script runs independently and processes videos ensuring the database is populated with the necessary data for efficient search and retrieval.
- **Integration and Workflow:**
 - The backend processing system prepares the data needed for the web application to function effectively.
 - When a user submits a query, the web application leverages the pre-processed data to find and display relevant video scenes quickly.